BetterCode: Shareable

Publish codes that others can use

Overview

- *Recap day 1*+*2*+*3*+4 - readable, reusable, reproducible, reliable
  - Help with being a better coder
  - Generate high-quality code
- **Shareable**: Github
  - Brag about your research
  - Activity 1: Publish Github repo
  - Activity 2: End-of-workshop quiz

You **should** and **must** share your code & data (to publish)

- Transparency
- Collaboration & communication
- Requirements of many major journals

CovidSim (Imperial College)

- Simulation model of COVID-19, influence major lockdown policies
- Released: March 2020
- Critism
  - Slow code release: April/May 2020
  - Messy code, legacy code
- Independent validation
  - Core conclusion holds
  - Workable
  - Largely reproducible

INFLUENTIAL PANDEMIC SIMULATION VERIFIED BY CODE CHECKERS

Model shown to be reproducible after software engineers called the underlying code 'a buggy mess'.

By Dalmeet Singh Chawla

"Totally unreliable." "A buggy mess." Over the past month, software engineers have sharply criticized the code underpinning an influential coronavirus simulation by scientists at Imperial College London, one of several models that helped to sway UK politicians into declaring a lockdown. Some media articles even suggested that the simulation couldn't be repeated by others – casting further doubt on the study. Now, a computational neuroscientist has reported that he has independently rerun the simulation and reproduced its results.

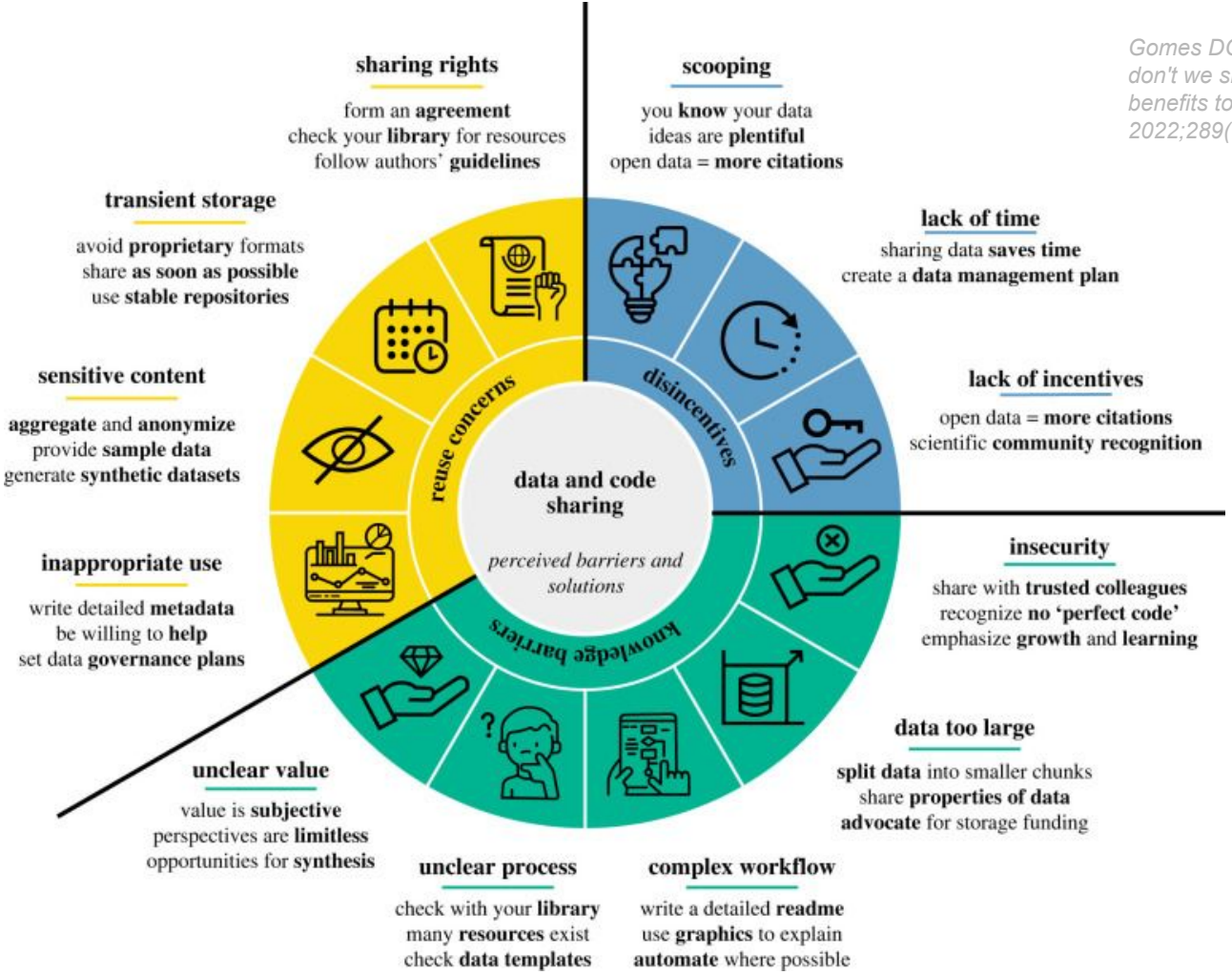
The successful code testing isn't a review of the scientific accuracy of the simulation, produced by a team led by mathematical epidemiologist Neil Ferguson. But it dispels some misapprehensions about the code, and shows that others can repeat the original findings.

The new test is "the best possible verification of Ferguson's simulations given the state of the art in computational science", says Konrad Hinsen, a computational biophysicist at the French national research agency CNRS in Paris, who was not involved in the work. In May, he wrote in a blogpost that the study's code looked "horrible", but that such shortcomings are expected in code written by scientists who aren't specialists in software development.

Released in mid-March, the original study reported that there could be half a million UK deaths if nothing were done to stop the virus, and modelled how policy interventions might help (N. Ferguson *et al.* 'Report 9' http://doi.org/ggqtdx; 2020). But Imperial scientists did not make the code available for public scrutiny. When a cleaned-up version was released

Nature | Vol 582 | 18 June 2020 | **323**

Singh Chawla, Dalmeet. "Critiqued coronavirus simulation gets thumbs up from code-checking efforts." Nature vol. 582,7812 (2020): 323-324. doi:10.1038/d41586-020-01685-y



Gomes DGE, Pottier P, Crystal-Ornelas R, et al. Why don't we share data and code? Perceived barriers and benefits to public archiving practices. Proc Biol Sci. 2022;289(1987):20221113. doi:10.1098/rspb.2022.1113

Share work (code) that you can be proud of

Internal Quality

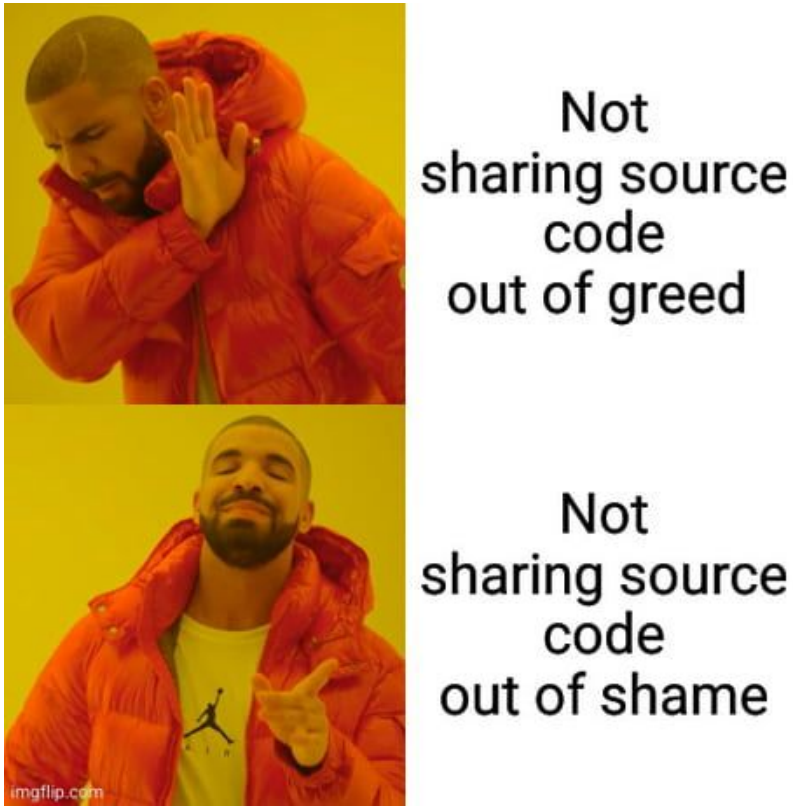
Readable

Reusable

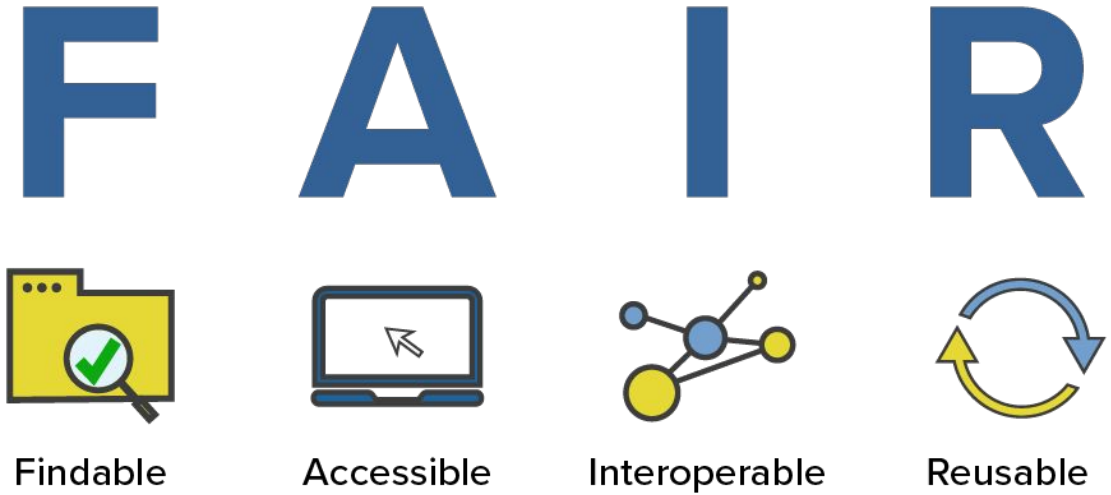
Reproducible

Reliable

Standard Compliance?



Sharing Data: FAIR



2016: FAIR
2021: FAIR4RS (FAIR For Research Software)

Sharing Code: TADA

Transferable
Available
Documented
Annotated

Preprint

Ivimey-Cook ER, et al. (2025). TADA! Simple guidelines to improve code sharing. EcoEvoRxiv. DOI:10.32942/X2D93K

MyCode.pdf

```
library(dplyr)
library(ggplot2)

data <-
read.csv("C:/mycomputer/caterpillar_data/data.csv")

summary_data <- data %>%
  group_by(habitat) %>%
  summarise(
    mean_count = mean(caterpillar_count),
    sd = sd(caterpillar_count),
  )

filtered_data <- data %>%
       filter(habitat != "D")

model1 <- glm(caterpillar_count ~ habitat,
  family = Poisson, data = filtered_data
)

figure1 <- ggplot(
  filtered_data,
  aes(x = habitat, y = catperillar_count)
) +
  geom_boxplot() +
  theme_bw()
```

Transferable
Available
Documented
Annotated

MyCode.R [T]

doi.org/... [A]

```
# Load packages#####
library(dplyr)
library(ggplot2)
library(here)

# Load caterpillar abundance data (w/o local file paths)#####
data <- read.csv(here("caterpillar_data", "data.csv"))

# summarise the mean number caterpillars with error#####
summary_data <- data %>%
  group_by(habitat) %>%
  summarise(
    mean_count = mean(caterpillar_count),
    sd = sd(caterpillar_count),
  )

# Remove values from habitat D as these are an error#####
filtered_data <- data %>%
       filter(habitat != "D")

# Run a Poisson general linear model######
# to analyse caterpillar abundance varying with habitat
# numeric results in "Caterpillar Abundance"
model1 <- glm(caterpillar_count ~ habitat,
  family = Poisson, data = filtered_data
)

#create figure 1, caterpillar count against habitat####
figure1 <- ggplot(
  filtered_data,
  aes(x = habitat, y = caterpillar_count)
) +
  geom_boxplot() +
  theme_bw()
```

[T]

[A]

README .txt [D]

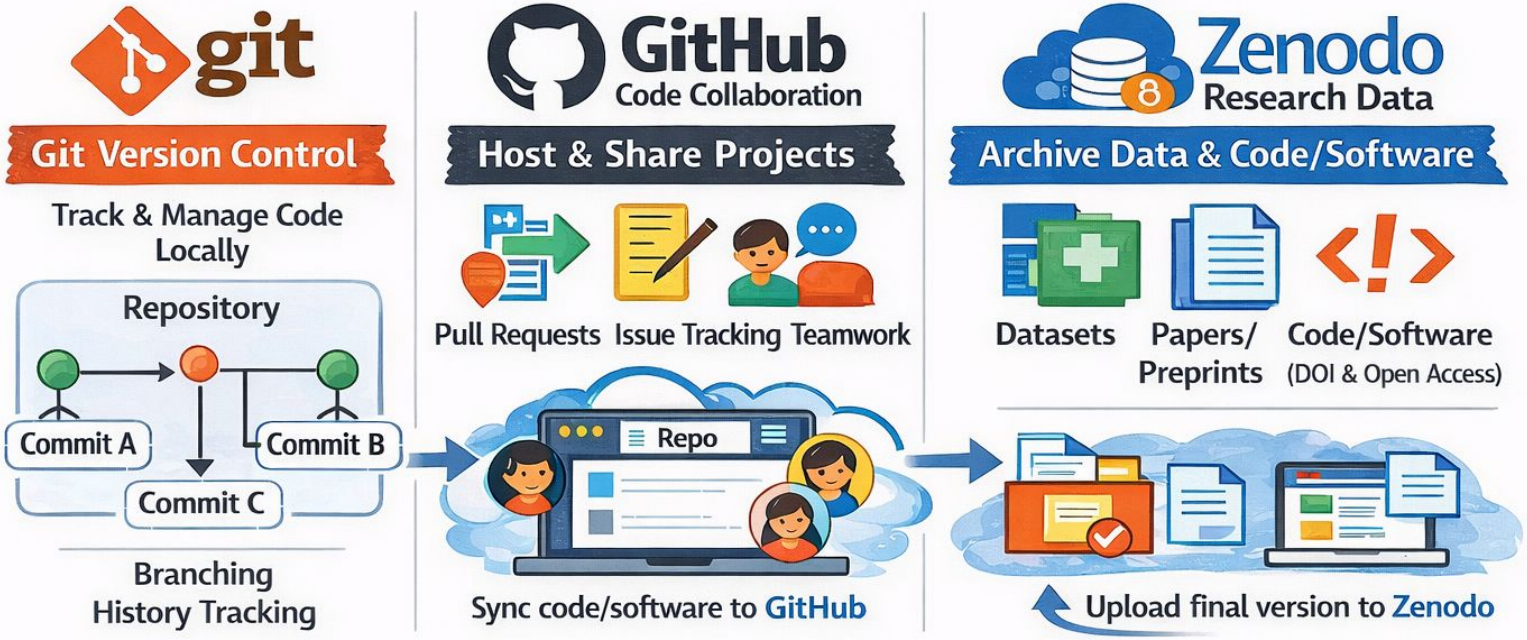
Authors: Ed Ivimey-Cook
Email: Ed@Ivimey-Cook.com
Title: Caterpillar abundance and habitat: A story.
Funders: SORTEE
Code License: MIT License in Code_License.txt

Code :
MyCode.R: Load packages, imports caterpillar_data, runs a poisson glm on filtered data. Produces Figure 1.

Software and Packages:
R v4.3.3
ggplot v2.3.2
dplyr v1.1.4
here v1.0.0

Data located here:
doi.org/...

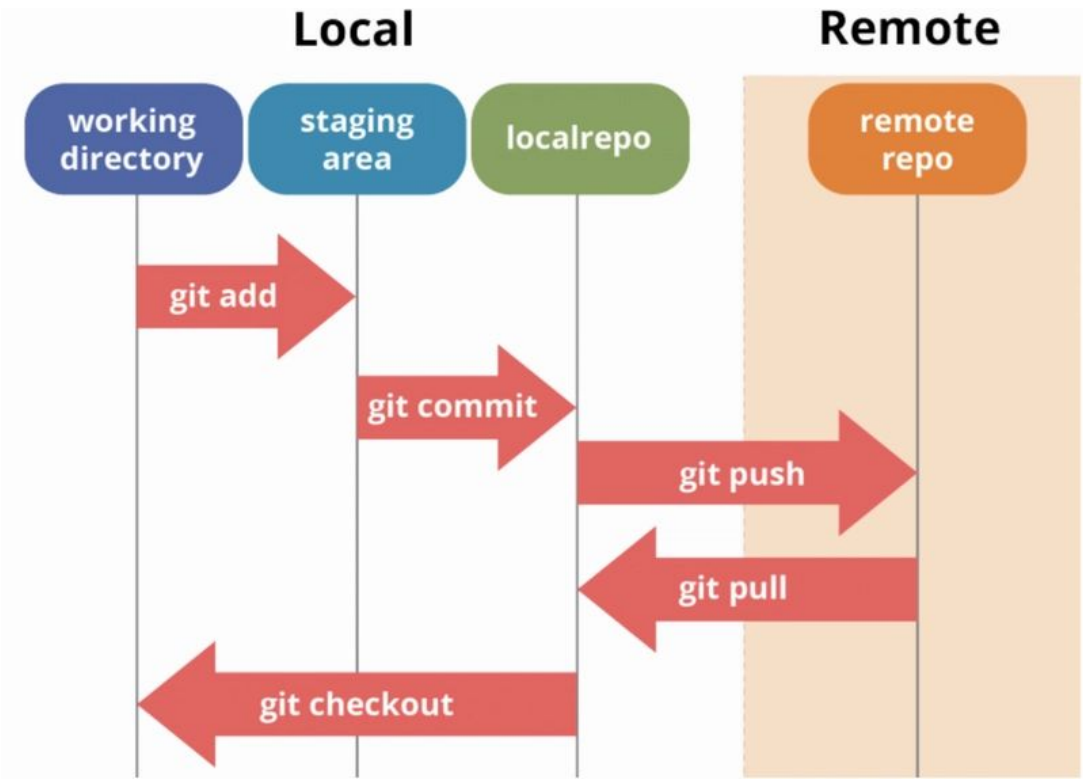
Understanding Git, GitHub & Zenodo



Demo 1: Local git - simple flow, no branch

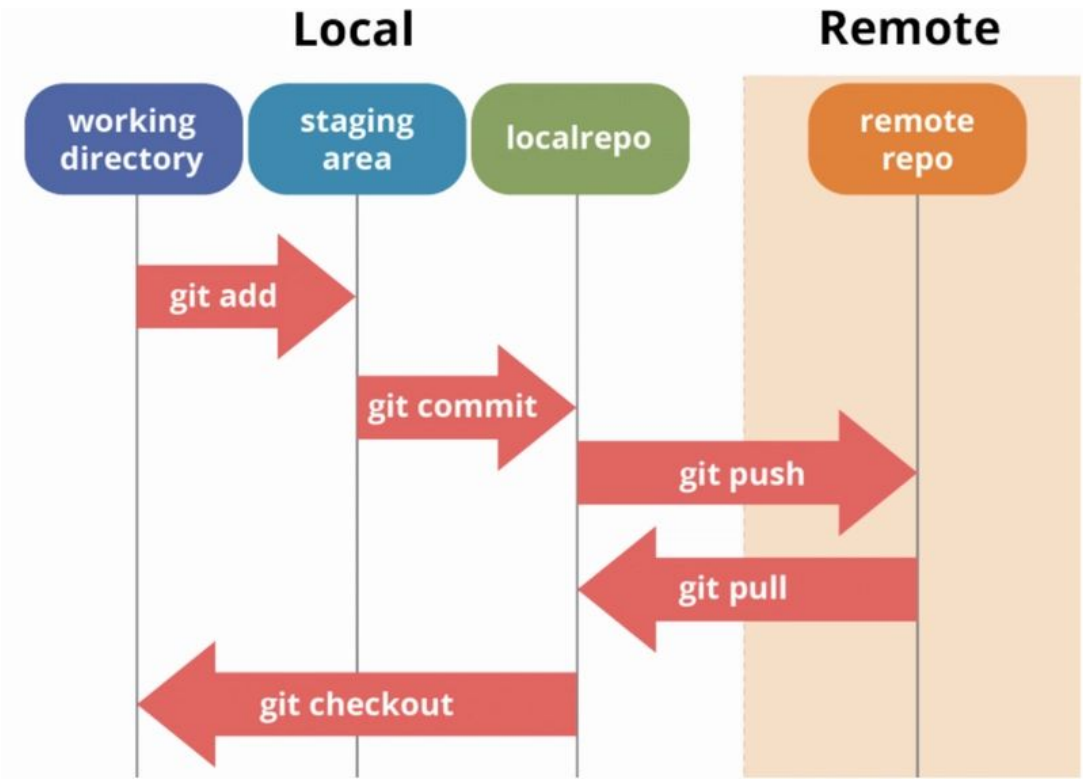
Check changes: git status
Check history: git log

The history is in the .git diretory



Demo 1: Github - remote repositories

Git remote add origin



Demo 1: Using IDE for version control with Git

Demo 1: Github Repo

- Good organization
- README.md
  - What the project does
  - How to use it
  - Example usage
  - Basic structure
- .gitignore
  - Include scripts and environment description
  - Once pushed, it’s saved in the history

Demo 1: Good Github Repo

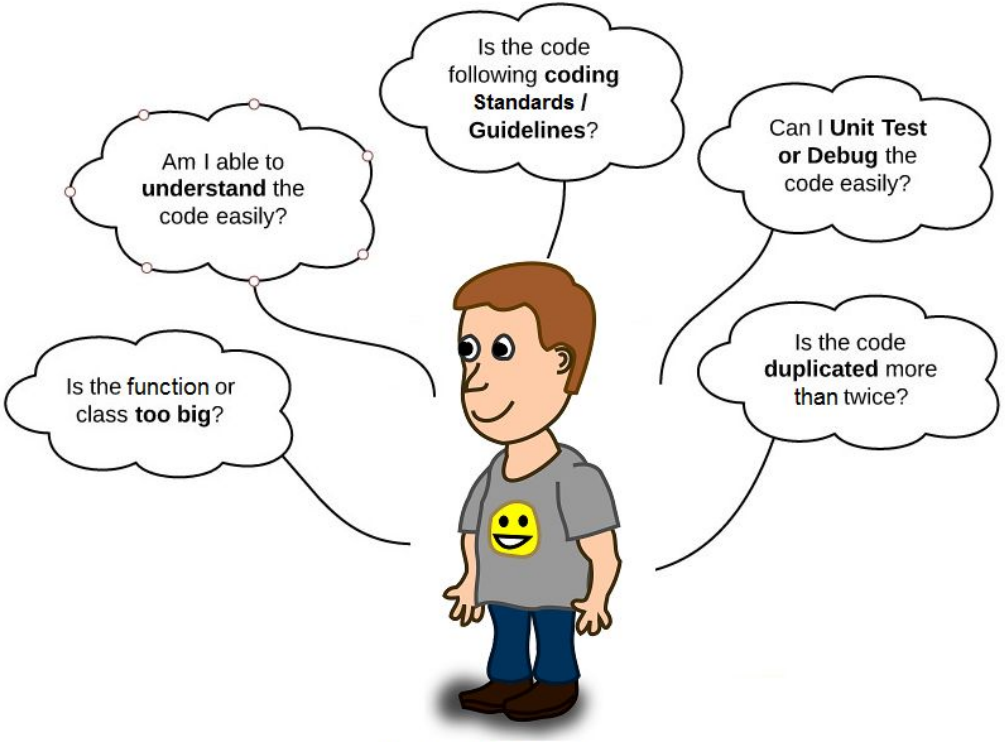
- Explain README.md, what should be included in
  - What the project does
  - How to use it
  - Example usage
  - Basic structure
  - Additional documents in /docs folder (if applicable)
- Explain .gitignore
- Explain versioning, note that deleting requires history clean up
  - Commit, push, pull
  - Do it within the IDE, on sumner2 too
- Show examples of real life Github repos: what is good, what is bad

Activity 1 - Publish your repo

- Create Github account
- Publish a directory
- Make changes
- Push, pull form it

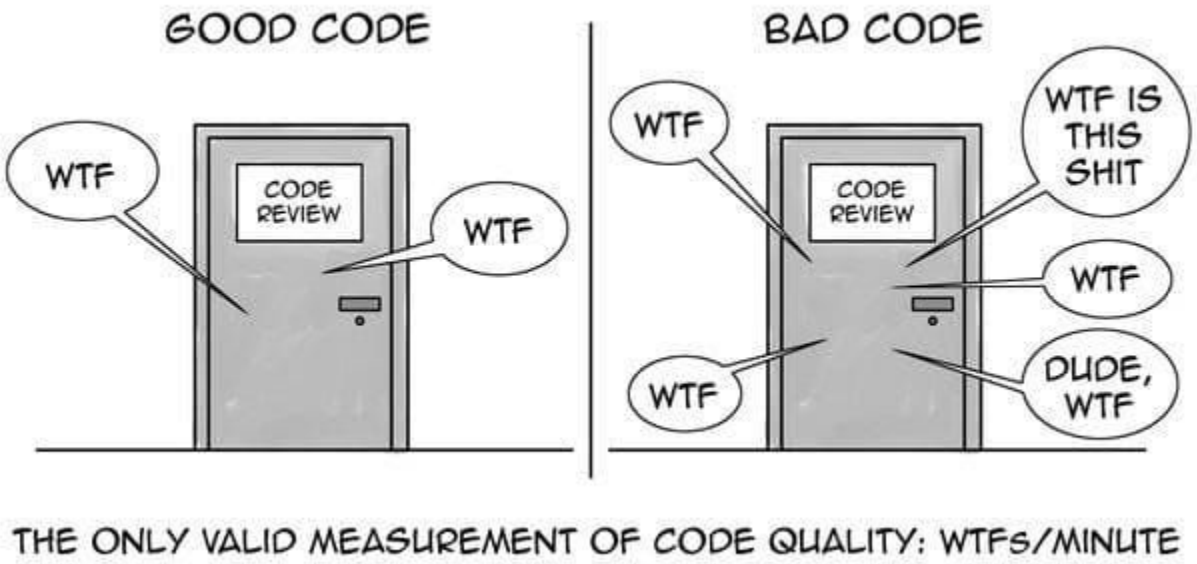
Make it a habit, so make it as convenience as possible!!

Code Review



Basic Code Reviewer

Demo 2 - Pull request



Workshop Quizzes

8 scenarios, some with multiple choices.

There is no single correct answer. With each option, think about when may they be right or when may they be wrong.

Scenario 1

A new collaborator opens your repo and they see:

```
analysis_final.py
analysis_final_v2.py
run_this.py
misc/
tmp/
```

A. Code is not optimzed
B. Missing comments in functions
C. Poor projects structure
D. File names are too long

What are the issues?

How should you fix it?

Scenario 2

You modify this functions over time. What's wrong with each version?

```r
calculation <- function(a, b, c, d) {
  x <- abs(c - a)
  y <- abs(d - b)
  return (x*y)
}
```

```python
def calculation(a, b, c, d):
    x = abs(c - a)
    y = abs(d - b)
    return (x*y)
```

Scenario 2

You modify this functions over time. What's wrong with each version?

```r
calculation <- function(a, b, c, d) {
  # Calculate the width
  x <- abs(c - a)
  # Calculate the height
  y <- abs(d - b)
  # Return the area (width x height)
  return (x*y)
}
```

```python
def calculation(a, b, c, d):
    # Calculate the width
    x = abs(c - a)
    # Calculate the height
    y = abs(d - b)
    # Return the area (width x height)
    return (x * y)
```

Scenario 2

You modify this functions over time. What's wrong with each version?

```r
calculate_area <- function(left, top, right, bottom) {
  width <- abs(right - left)
  height <- abs(top - bottom)
  area <- width * height
  return (area)
}
```

```python
def calculate_area(left, top, right, bottom):
    width = abs(right - left)
    height = abs(bottom - top)
    area = width * height
    return area
```

Scenario 3

What's wrong with this function? (Pay attention to global structure)

```r
clean_transform_plot <- function(df) {
  # Cleans data, applies transformations, and generates a plot

  # Cleaning
  df <- na.omit(df)
  # Imagine more cleaning code here
  df_cleaned <- df %>% filter(value > 0)

  # Transformation
  df_transformed <- df_cleaned
  # Imagine more transformation here
  df_transformed <- mutate(df_transformed, log_value = log(value))

  # Plotting
  p <- ggplot(df_transformed, aes(x = log_value, y = normalized)) +
    geom_point()

  return(p)
}
```

```python
def clean_transform_plot(df):
    """Cleans data, applies transformations, and generates a plot."""

    # Cleaning
    df = df.dropna()
    ## Imagine more cleaning code here
    df_cleaned = df[df["value"] > 0]

    # Transformation
    df_transformed = df_cleaned.copy()
    ## Imagine more transformation here
    df_transformed["log_value"] = np.log(df_transformed["value"])

    # Plotting
    fig, ax = plt.subplots()
    ax.scatter(df_transformed["log_value"], df_transformed["normalized"])
    plt.close(fig)

    return fig
```

Scenario 4

You have an analysis pipeline with 4 clear steps.

Cleaning → Transformation → Statistical Testing → Visualization

To rerun it with a similar dataset, you will:

A. Edit the code in your .R/.py and run on your IDE
B. Put the input path as a parameter in your notebook and rerun the entire notebook.
C. Turn the script into executable command line tool and run it
D. Ask ChatGPT to rerun it for you.

Scenario 5

Your workflow contains of three main components:

- A preprocessing steps with multiple C and Java-based tools
- Analysis 1: convenient to be conducted in python
- Analysis 2: convenient to be conducted in R

The project is in development, so there can be lots of daily changes.

What environment management should you use?

A. Container
B. conda-forge
C. uv (or other venv)
D. renv

Scenario 6

You finished running your analysis pipeline without an error. Later, you discover that your results do not align with results of another lab member.

You should

A. Rerun your analysis again, hoping for a miracle
B. Review your colleague’s code
C. Confirm the same environmental setup
D. Review intermediate outputs for quality control

Scenario 7

You want to write unit tests for your function. To write good unit tests, you should

```r
process_scores <- function(scores) {
  # Takes a list of scores, removes invalid values,
  # normalizes them to 0-1 scale, and returns the average

  # Step 1: Clean (remove negative or NA values)
  cleaned <- scores[!is.na(scores) & scores >= 0]

  # Step 2: Transform (normalize by max)
  max_val <- max(cleaned)
  normalized <- cleaned / max_val

  # Step 3: Aggregate (compute average)
  return(mean(normalized))
}
```

```python
def process_scores(scores):
    """
    Takes a list of scores, removes invalid values,
    normalizes them to 0-1 scale, and returns the average.
    """
    # Step 1: Clean (remove negative or None values)
    cleaned = [s for s in scores if s is not None and s >= 0]

    # Step 2: Transform (normalize by max)
    max_val = max(cleaned)
    normalized = [s / max_val for s in cleaned]

    # Step 3: Aggregate (compute average)
    return sum(normalized) / len(normalized)
```

A. Test them on large list
B. Check correctness at each step
C. Review the function docstring
D. Ask AI

Scenario 8

Your colleague asked you for your code. Because the analysis is still in progress, you sent them the path to your source code along with some instructions in Slack.

Two months later, your colleague messages: “I’ve made some changes. It was working, but now it breaks”.

What practice would have prevented this?

A. Say “no” to helping colleagues
B. Version your code early on Github and collaborate there
C. Send a zip file with a copy of your code
D. Write stricter instructions in your code (e.g. “DO NOT CHANGE”)

Food For Thought: Good Coding Practice in AI Era

Is coding & coding practice still important when AI writes most of the code?

	Human skills	How AI helps (not all)
Readable	Audience-awared communication	Suggest patterns Refactor Documentation
Reusable	Abstraction & tradeoff reasoning	
Reproducible	Root-cause debugging	Symptom debugging
Reliable	Intent testing & ownership mindset	Catch errors, automated testing
Shareable	Collab awareness & communication	Summary

Good coding habits trains you to decide what “good” looks like.
Using AI helps you get there fast.

Take-away messages

Better code makes your research understood, trusted, and adopted by others

- Readability = clear communication
- Reusability = consistency and extensibility
- Reproducibility = verifiable results
- Reliability = trustworthy results
- Sharing = adoption and impact